

Global Engineering Solutions

FEST®

FEST Library Reference Manual

1.	TComSerial		Page2
2.	Initialize		Page3
3.	CleanUp		Page4
4.	Connect		Page5
5.	DisConnect		Page6
6.	Send		Page7
7.	Recv		Page8
8.	ClearBuffer		Page10
9.	StartLog		Page11
10.	StopLog		Page12



1. TComSerial

● Description

- RS232 Port를 열고 닫는 부분으로 IO Handling Driver의 OnInitDevice() 부분에 삽입되어 진다.

● Syntax

```
TComSerial( int PortNum , int BaudRate , int DataBit , int StopBit , int ParityBit , bool DTRDSR=false , bool
            RTSCTS=false , bool XONXOFF=false );
```

```
TComSerial( TComHandler *pcComHandler , int PortNum , int BaudRate , int DataBit , int StopBit , int
            ParityBit , bool DTRDSR=false , bool RTSCTS=false , bool XONXOFF=false );
```

● Parameter

- PortNum: 열고자 하는 Port 번호
- BaudRate: 열고자 하는 Port 통신 속도
(256000/128000/115200/57600/56000/38400/19200/14400/9600/4800/2400/1200/600/300/110)
- DataBit: 열고자 하는 Port 의 Data Bit(5/6/7/8 중 하나)
- StopBit: 열고자 하는 Port 의 Stop Bit(1/2/3 중 하나, 1=1Bit, 2=2Bit, 3=1.5Bit를 의미)
- ParityBit: 열고자 하는 Port 의 Parity Bit
(0/1/2/3/4 중 하나, 0=No Parity, 1=Odd Parity, 2=Even Parity, 3=Mark Parity, 4=Space Parity를 의미)
- DTRDSR : 열고자 하는 Port 가 DTR/DSR를 사용할 지 결정
(0=DTR/DSR을 사용하지 않음을 의미하고, 0 이외의 값일 때는 DTR/DSR을 사용함을 의미)
- RTS/CTS : 열고자 하는 Port 가 RTS/CTS를 사용할 지 결정
0일 때는 RTS/CTS를 사용하지 않음을 의미하고, 1일 때는 RTS/CTS를 사용함을 의미하고, 그 이외의 값일 때는 RS485 Style의 RTS/CTS를 사용함을 의미한다(windows는 485를 지원하지 않으므로 의미 없음)
- XONXOFF : 열고자 하는 Port 가 XON/XOFF를 사용할 지 결정
(0=XON/XOFF를 사용하지 않음을 의미하고, 0 이외의 값일 때는 XON/XOFF를 사용함을 의미)

● Return

- 없음

● Sample

```
TComSerial *pcComSerial = new TComSerial(5, 9600 , 8 , 1 , 0 );
```

2. Initialize

- **Description**

- RS232 Port를 초기화 해주는 부분으로 IO Handling Driver의 OnInitDevice() 부분에 삽입되어 진다.

- **Syntax**

- bool Initialize();

- **Parameter**

- Nothing

- **Return**

- true : 성공, false : 실패

- **Sample**

- Initialize();

3. CleanUp

- **Description**

- 열려 있는 Port를 닫는다.

- **Syntax**

- bool CleanUp();

- **Parameter**

- Nothing

- **Return**

- true : 성공, false : 실패

- **Sample**

- CleanUp();

4. Connect

- **Description**

- RS232 Port를 연결 해주는 부분으로 IO Handling Driver의 OnInitDevice() 부분에 삽입되어 진다.

- **Syntax**

- bool Connect();

- **Parameter**

- Nothing

- **Return**

- true : 성공, false : 실패

- **Sample**

- Connect();

5. Disconnect

- **Description**

- 연결 되어 있는 Port를 끊는다

- **Syntax**

- `bool Disconnect();`

- **Parameter**

- Nothing

- **Return**

- true : 성공, false : 실패

- **Sample**

`Disconnect();`

6. Send

● Description

- Send Data를 RS232 Port에 Writing한다.

● Syntax

- `bool Send( const char* szSendData , int nNumOfBytesToSend );`

● Parameter

- `szSendData` : Writing하고자 하는 Data
- `nNumOfBytesToSend` : Writing하고자 하는 Data의 byte 수

● Return

- `true` : 성공, `false` : 실패

● Sample

```
BOOL Rs232_IO_Send( void* pDrvData, char *Cmnds ) {
    char SendBuf[DEFAULT_LENGTH];
    TComSerial *pComSerial;
    pComSerial = (TComSerial *)pDrvData;
    //
    sprintf( SendBuf , "%s%s" , Cmnds , SendTermStr );
    if ( !(pComSerial->Send(SendBuf, strlen(SendBuf))) ) {
        printf( "=====> [%s-DRIVER] Rs232_IO_Send() Error : %s\n" , DRIVER_NAME, SendBuf );
        return FALSE;
    }
    return TRUE;
}
```

7. Recv

● Description

- RS232 Port의 Read Buffer의 Character를 정해진 방식대로 읽는다.
읽은 Data는 szRecvBuf에 저장되며 szDelimiter와 일치되는 String을 만날 때 까지 Data를 읽게 되는데 nNumOfByteToRecv이상은 읽을 수 없다. nNumOfByteToRecv만큼 읽는 동안 szDelimiter과 일치하는 String을 만나지 못했을 때는 FALSE로 Return되며, TimeOut(milisec) 이내에 szDelimiter와 일치하는 String을 만나지 못했을 경우도 FALSE로 Return 된다.
szRecvBuf에 읽혀진 Data중 실제 읽힌 Data의 byte수는 nNumOfBytesTransferred 로 알 수 있게 된다.
TRUE로 결과가 Return되는 경우는 nNumOfByteToRecv 를 넘기지 않고 TimeOut시간 이내에 szDelimiter 과 일치되는 String을 만났을 때이다.

● Syntax

- bool Recv(char* szRecvBuf , const char* szDelimiter , int nNumOfByteToRecv , int* nNumOfBytesTransferred , int TimeOut);
- bool Recv(char* szRecvBuf , int nNumOfByteToRecv , int *nNumOfBytesTransferred , int TimeOut);

● Parameter

- szRecvBuf: Reading 되어진 Character Data
- szDelimiter: Character Data로서 Null로 끝나는 String이며, 이 String과 일치하는 String을 만날 때 까지 Data를 읽는다.
- nNumOfByteToRecv: 읽을 수 있는 최대 Character Data의 byte 수
- nNumOfBytesTransferred: 실제 읽혀진 Data의 byte 수
- TimeOut : Data를 읽을 수 있는 TimeOut시간(msec)

● Return

- true : 성공, false : 실패

● Sample

```
BOOL Rs232_IO_Recv( void* pDrvData, char *Response ) {
    int ReadCnt;
    BOOL Result;
    TComSerial *pComSerial;
    pComSerial = (TComSerial *)pDrvData;
    //
    Result = pComSerial->Recv(Response, RecvTermStr, 255, &ReadCnt, TimeOut);
    if ( Result==FALSE ) {
        printf( "=====> [%s-DRIVER] Rs232_IO_Recv() Error : [%s]\n" , DRIVER_NAME, Response );
    }
    return Result;
}
```

8. ClearBuffer

- **Description**

- RS232 Port(Read Buffer)에 도착해 있으나 읽지 않은 Data를 모두 지워버린다

- **Syntax**

- void ClearBuffer();

- **Parameter**

- Nothing

- **Return**

- Nothing

- **Sample**

ClearBuffer();

9. StartLog

● Description

- Port Logging을 위한 File 경로 및 이름과 어떤 형식으로 저장할지를 정의하며, Logging을 시작한다.

● Syntax

- void StartLog(const char* szLogFile , bool fAscii);

● Parameter

- szLogFile : 로깅할 파일의 경로 및 이름
- fAscii : TRUE is Ascii type logging, FALSE is Binary type logging

● Return

- Nothing

● Sample

```
SYSTEMTIME SysTime;
char          FileName[256];
TComSerial *pcComSerial;
pcComSerial = (TComSerial *)pDrvData;
GetLocalTime( &SysTime );
sprintf( FileName , "Datalog\\%s\\Driver\\%s\\Robot_COM%d_%04d%02d%02d%02d.log" ,
        CommPortNum, SysTime.wYear , SysTime.wMonth , SysTime.wDay , SysTime.wHour );
pcComSerial->StartLog(FileName , TRUE );
```

10. StopLog

- **Description**

- Port Logging을 멈춘다

- **Syntax**

- ```
void StopLog();
```

- **Parameter**

- Nothing

- **Return**

- Nothing

- **Sample**

- ```
TComSerial *pcComSerial;  
pcComSerial = (TComSerial *)pDrvData;  
pcComSerial-> StopLog( );
```

URL : <http://www.gesolutions.co.kr/>

(주)게스

우 305-500

대전광역시 유성구 용산동 533-1번지 미건테크노월드Ⅱ C동 440호

전화 : 042-671-3331

G.E.S

#440C-Gate Migun TechnoWorld Ⅱ, 533-1

Yongsan-dong, Yuseong-gu, Daejeon, Korea

Tel : +82-42-671-3331

